

1주 차 도구·개념 정리

3개월 자기주도 BE 전환 프로그램 · 1주 차(2026-09-08 ~ 09-10) 보조자료 · 백엔드 처음 시작하는 C#/Unity 개발자용

이 문서는 이번 주 세션에서 "시키는 대로 따라 했지만 익히지는 못한" 것들을 다시 정리한 것이다. 새 내용은 없다. 전부 이번 주에 한 번 이상 실제로 손으로 친 것이다. 각 항목에 **필수** 또는 **알아두기** 표시를 붙였다. 필수는 다음 주부터 매일 쓰는 것이고, 알아두기는 "이런 게 있다"만 알면 되는 것이다.

이번 주에 배운 것을 한 문장으로. 코드를 고치고(편집기) → 사진을 찍어 보내고(Git) → 남에게 보여 주고(GitHub PR) → 기계가 검사하고(CI) → 합친다(merge). 이 다섯 단계가 앞으로 매일 도는 루프다. 아래 모든 도구는 이 루프의 어느 한 칸에 속한다.

단계	쓰는 도구	이번 주 실제로 한 것
고친다	VS Code, 터미널	<code>ci.yml</code> , <code>docker-compose.yml</code> , PR 설명 작성
빌드·테스트한다	JDK, Gradle, JUnit	<code>./gradlew build</code> , 테스트 일부러 깨뜨리기
사진 찍어 보낸다	Git	add → commit → push, 브랜치, revert
보여 준다	GitHub, gh	PR #1, #2 열기, 리뷰 반영, 병합
기계가 검사한다	GitHub Actions	CI 초록·빨강 확인, main 보호 규칙
DB를 띄운다	Docker, Compose	MySQL 9.7 컨테이너 (별도 문서 <code>docker-1단계-개념</code>)

1. 터미널 **필수**

맥의 기본 셸은 zsh다. 이번 주에 친 명령은 전부 이 안에서 돌았다. Unity에서 GUI 버튼으로 하던 일을 글자로 시키는 것이라고 보면 된다. 자주 쓴 것만 정리한다.

명령	뜻	이번 주 예
<code>cd 경로</code>	폴더 이동	<code>cd docker</code>
<code>ls</code> , <code>ls -la</code>	폴더 내용 보기. <code>-la</code> 는 숨김 파일·권한까지	<code>ls gradle/wrapper/</code>
<code>cat 파일</code>	파일 내용을 화면에 출력	<code>cat .github/workflows/ci.yml</code>
<code>mkdir -p a/b</code>	폴더 만들기. <code>-p</code> 는 중간 폴더까지 한 번에	<code>mkdir -p .github/workflows</code>
<code>code 파일</code>	VS Code로 열기. 없는 파일이면 새 파일	<code>code .github/workflows/ci.yml</code>
<code>A && B</code>	A가 성공하면 B 실행. A가 실패하면 B는 건너뛴	<code>git commit ... && git push ...</code>
<code>A B</code>	A의 출력을 B의 입력으로 넘김	<code>docker compose logs mysql grep ready</code>
<code>grep 문자열</code>	입력에서 문자열이 있는 줄만 골라냄	위 예
<code>brew install X</code>	맥 패키지 관리자 Homebrew로 설치	<code>brew install gradle, gh</code>
위 화살표	직전 명령 다시 불러오기	실패한 <code>gh pr create</code> 재실행
Ctrl+C	지금 도는 명령 중단	<code>gh pr checks --watch</code> 빠져나오기

헛갈릴 때 기준: 경로에서 `.` 은 현재 폴더, `..` 은 상위 폴더, `~` 는 홈 폴더(`/Users/dreadwitch`). `./gradlew` 의 `./` 는 "현재 폴더에 있는 gradlew를 실행"이라는 뜻이다.

2. JDK와 Java 용어 필수

C#에서 .NET SDK와 런타임이 있듯, Java에는 JDK와 JVM이 있다.

용어	뜻	C# 대응
JVM	컴파일된 Java 프로그램(<code>.class</code>)을 실행하는 가상 머신	.NET 런타임(CLR)
JDK	JVM + 컴파일러(<code>javac</code>) + 도구. 개발할 때 설치하는 것	.NET SDK
Temurin	JDK를 만드는 배포판 이름 중 하나(Eclipse 재단). 우리가 설치한 것	—
21 LTS	장기 지원 버전. 이 프로젝트가 고정한 Java 버전	.NET 8 LTS 같은 것
<code>JAVA_HOME</code>	"어떤 JDK를 쓸지" 가리키는 환경변수. <code>~/.zshrc</code> 에 21로 고정함	—
toolchain	<code>build.gradle.kts</code> 에 적힌 "컴파일은 무조건 21로"라는 설정. <code>JAVA_HOME</code> 이 26을 가리켜도 산출물은 21	csproj의 <code><TargetFramework></code>

이번 주 겪은 혼란. `brew install gradle` 이 JDK 26을 같이 깔았다. 그래서 두 가지가 생겼다. (1) `JAVA_HOME` 은 "Gradle 자체를 띄우는 JVM"을 정한다. (2) `toolchain`은 "내 코드를 컴파일·테스트하는 JDK"를 정한다. 둘은 다른 것이고, 코드 산출물에 중요한 건 (2)다.

3. Gradle 필수

Gradle은 빌드 도구다. "소스를 컴파일하고, 테스트를 돌리고, 실행 파일을 만들어라"를 대신 해 준다. C#에서는 Visual Studio나 `dotnet build` 가 `csproj`를 읽고 이 일을 했다.

3.1 프로젝트 파일들

파일	역할	C# 대응
<code>settings.gradle.kts</code>	이 프로젝트 이름과 하위 모듈 목록. 우리는 <code>app</code> 하나	<code>.sln</code>
<code>app/build.gradle.kts</code>	이 모듈의 빌드 설정. 플러그인, 의존성, <code>toolchain</code> , 실행 클래스	<code>.csproj</code>
<code>gradle/libs.versions.toml</code>	의존성 이름과 버전을 한곳에 모아둔 목록	<code>Directory.Packages.props</code>
<code>gradle.properties</code>	Gradle 동작 옵션(메모리 등)	—
<code>gradlew</code> , <code>gradlew.bat</code> , <code>gradle/wrapper/*</code>	wrapper . 아래 3.2	Unity Hub + <code>ProjectVersion.txt</code>
<code>app/src/main/java/...</code>	실제 코드	<code>Assets/Scripts</code>
<code>app/src/test/java/...</code>	테스트 코드	테스트 프로젝트
<code>app/build/</code>	빌드 결과물. git에 넣지 않음 (<code>.gitignore</code>)	<code>bin/</code> , <code>obj/</code>

`.kts` 는 Kotlin 문법으로 쓴 설정 파일이라는 뜻이다. Groovy 문법 (`.gradle`)도 있는데 우리는 Kotlin DSL을 골랐다. 문법을 외울 필요는 없고 "설정 파일이 코드처럼 생겼다" 정도만 알면 된다.

3.2 wrapper 필수

wrapper는 `gradle` 본체가 아니라 "정해진 버전의 `gradle`을 내려받아 실행해 주는 작은 실행기"다. 프로젝트에 같이 commit된다. 네 파일이다.

- `gradlew` : 맥·리눅스용 실행 스크립트. 우리가 치는 `./gradlew` 가 이것
- `gradlew.bat` : 윈도우용. 같은 일
- `gradle/wrapper/gradle-wrapper.jar` : 실제로 다운로드·실행하는 작은 자바 프로그램

- `gradle/wrapper/gradle-wrapper.properties` : 어느 버전을 어디서 받을지. 우리는 `gradle-9.7.1-bin.zip`

`./gradlew build` 를 치면: 스크립트가 jar 실행 → jar가 properties를 읽음 → `~/.gradle/wrapper/dists/` 에 9.7.10이 있으면 쓰고 없으면 다운로드 → 그 gradle에 `build` 를 넘김.

왜 중요한가. 팀원마다, CI 러너마다 gradle 버전이 달라도 `./gradlew` 만 치면 모두 9.7.1로 빌드한다. 그래서 CI 워크플로에 "gradle 설치"가 없다. 로그의 `Downloading .../gradle-9.7.1-bin.zip` 줄이 그 증거였다. 시스템에 `brew install gradle` 로 깐 gradle은 `gradle init` 한 번에만 쓰고 그 뒤로는 쓰지 않는다.

3.3 태스크 필수

명령	하는 일
<code>./gradlew build</code>	컴파일 + 테스트 + jar 생성. 테스트가 실패하면 build도 실패. CI가 도는 명령
<code>./gradlew test</code>	테스트만
<code>./gradlew run</code>	<code>application { mainClass }</code> 에 적힌 클래스 실행
<code>./gradlew clean</code>	<code>build/</code> 폴더 삭제. "처음부터 다시"를 확인할 때 <code>clean build</code>

결과는 마지막 줄로 본다. `BUILD SUCCESSFUL` 또는 `BUILD FAILED`. 실패하면 그 위에 어느 태스크(`:app:test` 등)가 왜 실패했는지 나온다. 테스트 상세는 `app/build/reports/tests/test/index.html`.

3.4 JUnit 알아두기

Java 테스트 프레임워크. C#의 NUnit/xUnit에 해당. `@Test` 는 `[Test]` 어트리뷰트와 같고, `assertNotNull(...)` 은 `Assert.NotNull(...)` 이다. 지금은 샘플 테스트 1개(`AppTest.appHasAGreeting`) 뿐이고, 이번 주에는 "테스트 파이프라인이 도는지 확인하는 용도"로만 썼다. 2주 차에 실제 테스트를 쓰기 시작한다.

4. Git 필수

이번 주 가장 많이 실수한 곳이다. 그림 하나로 정리한다.

[작업 트리]	--add-->	[스테이지]	--commit-->	[로컬 저장소]	--push-->	[원격(GitHub)]
내가 편집한 파일		사진에 넣을 파일 고르기		찍힌 사진들 (commit 목록)		GitHub에 올라간 사진들

- **add = 고르기.** "이 파일을 다음 사진에 넣겠다."
- **commit = 사진 찍기.** 고른 파일들의 현재 상태를 하나로 묶어 저장. 메시지를 붙인다.
- **push = 사진 보내기.** 로컬에 쌓인 commit을 GitHub로. **commit 안 한 건 push되지 않는다.** 이번 주 두 번 겪었다.

4.1 git status --short 읽기 필수

표시	뜻	다음 할 일
?? 파일	git이 모르는 새 파일	git add
빨간 M (오른쪽 열)	수정했지만 아직 고르지 않음	git add
초록 M 또는 A (왼쪽 열)	골라 둬. commit 대기	git commit
아무것도 없음	깨끗함. 작업 트리 = 마지막 commit	—

4.2 매일 쓰는 명령

명령	뜻
git status --short	지금 뭐가 바뀌었나. 모든 git 작업 전후에 친다
git add 파일	고르기
git commit -m "메시지"	사진 찍기. 메시지는 BE-002: ... 또는 docs: ... 형식
git push origin 브랜치	보내기. 새 브랜치 첫 push는 -u 붙임
git log --oneline -3	최근 commit 3개 한 줄씩
git log origin/브랜치 --oneline -1	원격에 올라간 마지막 commit. push 확인용
git diff	add 전 변경 내용. --cached 를 붙이면 add 후 내용
git switch -c 브랜치	새 브랜치 만들고 이동. 티켓마다 하나
git switch main	main으로 이동
git pull	원격의 새 commit을 가져와 합침. 병합 후 main에서
git revert HEAD --no-edit	마지막 commit을 "취소하는 새 commit" 생성. 이력이 남음

4.3 용어

- **브랜치**: commit 줄기의 이름. main 은 기준 줄기, BE-001A 는 티켓 작업 줄기. Unity로 치면 싼을 복제해서 실험 하는 것과 비슷하되, 나중에 합칠 수 있다.
- **origin**: 원격 저장소의 별명. origin/main 은 "마지막으로 GitHub와 통신했을 때 본 원격 main".
- **HEAD**: 지금 내가 서 있는 commit. 보통 현재 브랜치의 마지막 commit.
- **merge**: 브랜치를 main에 합치기. 우리는 **일반 merge**(commit 이력 보존)를 쓴다. squash는 여러 commit을 하나로 뭉치는데, 기록이 해시를 증거로 쓰므로 안 쓴다.
- **revert vs reset**: revert는 취소 commit을 추가(안전, 이력 남음). reset은 이력을 지움(이번 주 안 씀, 당분간 쓰지 말 것).

5. GitHub와 gh 필수

Git은 내 컴퓨터의 도구, GitHub는 그 저장소를 올려 두는 웹 서비스다. `gh`는 GitHub를 터미널에서 다루는 명령어(GitHub CLI)다. 웹에서 클릭으로 하는 일을 글자로 한다.

5.1 PR(Pull Request)

"내 브랜치를 main에 합쳐 달라"는 요청서. 코드 차이(diff), 설명, 리뷰 댓글, CI 결과가 한 화면에 모인다. 이번 주 PR #1(BE-001), PR #2(BE-001A)를 열었다. PR 설명은 `docs/pr/티켓.md`에 네 항목으로 쓴다: ① 구현 내용 ② 설계 결정과 이유 ③ 테스트 방법과 결과 ④ 한계와 확인받고 싶은 것.

PR 설명에서 두 번 지적받은 것. "AI 안내에 따라 했다"는 이유가 아니다. 리뷰어가 알고 싶은 것은 내가 왜 이렇게 골랐는지와 실제로 돌린 명령과 그 출력이다. 파일 경로, 명령, 출력 한 줄 인용이 있으면 통과한다.

5.2 gh 명령

명령	뜻
<code>gh auth login</code>	로그인(1회 완료)
<code>gh pr create --base main --head 브랜치 --title "..."</code> <code>--body "..."</code>	PR 열기. 옵션 없이 치면 대화형으로 물어봄
<code>gh pr checks --watch</code>	이 브랜치 PR의 CI 결과를 실시간으로. push 직후엔 몇 초 기다렸다가
<code>gh pr view 번호</code>	PR 내용 보기. <code>--web</code> 이면 브라우저로
<code>gh pr merge 번호 --merge --delete-branch</code>	일반 merge로 병합하고 브랜치 정리. 로컬은 main으로 돌아옴
<code>gh run list</code> , <code>gh run view ID --log</code>	CI 실행 목록과 로그

6. GitHub Actions와 CI 필수

CI(Continuous Integration): 코드를 올릴 때마다 **자동으로** 빌드·테스트해서 깨졌는지 확인하는 장치. Unity Cloud Build와 같은 개념. **GitHub Actions**는 그걸 돌려 주는 GitHub의 서비스.

6.1 워크플로 파일 한 줄씩

```
name: CI                                # 이 지시서의 이름. PR 화면에 표시

on:                                     # 언제 돌릴지
  push:
    branches: ["main"]                 # main에 commit이 올라오면
  pull_request:
    branches: ["main"]                 # main을 향한 PR이 열리거나 갱신되면

jobs:                                   # 무엇을 돌릴지
  build:                                # 잡 이름(자유). Ruleset에서 이 이름을 검사 항목으로 씀
    runs-on: ubuntu-latest             # 우분투 리눅스 컴퓨터를 빌려서
    steps:                              # 순서대로
      - uses: actions/checkout@v6      # 1) 저장소 내려받기 (남이 만든 기능 = uses)
      - uses: actions/setup-java@v5    # 2) JDK 설치
        with:                           # 그 기능에 넘기는 옵션
          java-version: '21'
          distribution: 'temurin'
      - run: ./gradlew build            # 3) 명령 직접 실행 (= run)
```

- **러너**: 매번 새로 빌리는 깨끗한 컴퓨터. 내 맥에 칸 건 거기 없다. 그래서 JDK는 설치하고, gradle은 wrapper가 받아 오니 안 칸다.
- **uses vs run**: `uses` 는 GitHub 마켓의 기성 기능(액션)을 가져다 씀. `run` 은 터미널 명령을 그대로 씀.
- **@v5** : 액션의 버전. 한국어 문서는 `@v4` 였는데 실행해 보니 deprecated 경고가 떠서 v5로 올렸다. **문서보다 실제 실행 결과를 믿는다.**

6.2 언제 도는가

상황	CI	이유
PR 없이 작업 브랜치에 push	안 돔	<code>on</code> 의 두 조건 모두 해당 없음
main 대상 PR 열기	돔	<code>pull_request</code>
PR 열린 상태에서 그 브랜치에 push	돔	PR이 갱신되어 <code>pull_request</code> 다시 발생
PR 병합	돔	main에 commit 생김 → <code>push</code>

6.3 Ruleset(브랜치 보호) 필수

CI는 결과를 **보여 줄 뿐** 막지는 않는다. 막으려면 저장소 설정(Settings → Rules → Rulesets)에서 규칙을 건다. 우리 `main` 보호 규칙: (1) PR을 통해서만 main에 합칠 수 있다 (2) `build` 검사가 통과해야 합칠 수 있다 (3) main 삭제 금지 (4) 강제 push 금지. 처음 만들 때 "대상 브랜치"를 안 넣어서 아무 데도 안 걸린 상태였다. 규칙은 "무엇을"과 "어디에" 둘 다 있어야 한다.

이번 주 실험의 의미. 초록불만 봐서는 CI가 일하는지 모른다. 테스트를 일부러 깨뜨려 push → 빨간불 → revert → 초록불. 이 세 번이 "실패를 잡는다"는 증거다. 앞으로 어떤 자동화든 **일부러 실패시켜 보는 것**이 검증 방법이다.

7. YAML 필수

`docker-compose.yml` 과 `ci.yml` 둘 다 YAML이다. 앞으로 Spring 설정(`application.yml`)도 YAML이다. 규칙은 넷뿐이다.

1. **맵(키: 값)**: `name: CI`. 콜론 뒤 공백 필수. 콜론 앞엔 공백 없이.
2. **중첩은 들여쓰기**: 스페이스 2칸씩. **탭 금지**. 같은 깊이는 같은 칸수.
3. **리스트**: `- 항목`. `steps:` 아래 각 스텝, `branches: ["main"]` 의 대괄호도 리스트.
4. **문자열 따옴표**: 보통 생략 가능. 숫자처럼 보이는 걸 문자열로 두고 싶으면 `'21'` 처럼 감싼다.

이번 주 오류는 전부 "맵이어야 할 곳에 리스트" 또는 "들여쓰기 한 칸 차이"였다. 오류 메시지의 `must be a mapping` 은 "여기 맵이 와야 한다", `must be a array` 는 "여기 리스트가 와야 한다"는 뜻이다.

8. Docker와 Compose 알아두기

개념은 `docker-1단계-개념.html` 에 있으니 여기서는 명령만 적는다. 2주 차 이후 MySQL이 필요할 때 다시 쓴다.

명령 (<code>docker/</code> 폴더에서)	뜻
<code>docker compose up -d</code>	컨테이너 기동(백그라운드). 재기동도 이것
<code>docker compose ps</code>	상태 확인
<code>docker compose logs mysql</code>	로그. <code>ready for connections</code> 가 보이면 준비됨
<code>docker compose down</code>	컨테이너 종료·삭제. 볼륨(데이터)은 남음
<code>docker compose config</code>	yml 문법 검사

비밀번호는 `docker/.env` 에 있고 git에 올라가지 않는다(.gitignore). 흐름: `.env` → compose가 `${VAR}` 를 치환 → 컨테이너 환경변수 → MySQL 첫 초기화 때 사용.

9. 프로젝트 운영 파일 알아두기

파일	누가 쓰나	언제
LEARNING_STATE.md	튜터가 갱신, 학습자가 읽음	매 세션 시작·종료. "지금 어디까지 왔나"
VERSIONS.md	튜터·학습자	버전 고정 근거. 왜 Java 21, MySQL 9.7인지
docs/notes/날짜.md	학습자	매일 종료 시. 7개 항목
docs/pr/티켓.md	학습자	PR 열 때. 4개 항목
docs/reviews/티켓-review.md	Reviewer	PR 리뷰. 반영 요청 표

10. 우선순위 정리

다음 주 월요일까지 손에 익혀야 하는 것 (외우기가 아니라, 보지 않고 칠 수 있게).

1. `git status --short` → `git add` → `git commit -m` → `git push`. 그리고 status 색 읽기.
2. `git switch -c` 브랜치로 시작해서 `gh pr create` → `gh pr checks --watch` → `gh pr merge --merge`로 끝나는 티켓 한 바퀴.
3. `./gradlew build` 결과 마지막 줄 읽기. 실패하면 어느 태스크가 왜 실패했는지 찾기.
4. YAML 들여쓰기 규칙 넷.

설명할 수 있으면 되는 것. wrapper가 뭔지와 왜 CI에 gradle 설치가 없는지. toolchain과 JAVA_HOME의 차이. CI가 언제 도는지(6.2 표). Ruleset이 CI와 어떻게 다른지.

지금은 몰라도 되는 것. Kotlin DSL 문법, JUnit 상세, Docker 내부 구조, gradle 캐시·concurrency 같은 CI 최적화, `git reset/rebase`.

확인 질문 (답을 적은 뒤 튜터에게 보여 준다)

1. `git status --short` 에서 빨간 M 과 초록 M 의 차이는? 각각 다음에 칠 명령은?

2. commit 없이 push하면 무슨 일이 생기나? 터미널에 뭐라고 나오나?

3. `./gradlew build` 를 쳤을 때 gradle이 컴퓨터에 없으면 어떻게 되나? 어느 파일이 관여하나?

4. CI 워크플로에 JDK 설치는 있는데 gradle 설치는 없다. 이유는?

5. 작업 브랜치에 push했는데 CI가 안 돌아왔다. 고장인가? 언제 도는가?

6. CI가 빨간불인데 Merge 버튼이 눌린다. 무엇이 빠진 것인가?

7. `JAVA_HOME` 이 JDK 26을 가리키는데 산출물은 21로 나온다. 무엇이 그렇게 만드나?

8. YAML에서 `must be a mapping` 오류가 났다. 무엇을 확인하나?

9. squash merge를 안 쓰기로 한 이유는?

10. PR 설명 ②에 "AI 안내대로 했다"고 쓰면 왜 부족한가? 무엇을 써야 하나?